HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN: PHÂN TÍCH VÀ KHAI PHÁ DỮ LIỆU VĂN BẢN

XÂY DỰNG HỆ THỐNG PHÂN LOẠI SPAM/HAM CHO BÌNH LUẬN SINH VIÊN PTIT TRÊN FACEBOOK

Giảng viên hướng dẫn	……
Lớp/nhóm	……
Thành viên 1	Cương — MSSV: ……
Thành viên 2	Bảo — MSSV: ……
Thành viên 3	Phi — MSSV: ……
Thành viên 4	Phương — MSSV: ……

HÀ NỘI, THÁNG 11 NĂM 2026

MỤC LỤC

MỞ ĐẦU 1

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT 2

1.1. Bài toán phân loại văn bản . . . 2

1.1.1. Học có giám sát và phân loại nhị phân . . . 2

1.1.2. Đặc điểm bình luận tiếng Việt . . . 2

1.1.3. Gán nhãn có hỗ trợ AI . . . 2

1.2. Làm sạch và tiền xử lý văn bản . . . 3

1.2.1. Chuẩn hóa và bảo toàn dữ liệu . . . 3

1.2.2. Tách từ và từ dừng . . . 3

1.2.3. Tín hiệu đặc biệt và giới hạn che tên . . . 3

1.3. Các phương pháp biểu diễn văn bản . . . 4

1.3.1. Bag-of-Words . . . 4

1.3.2. TF-IDF . . . 4

1.3.3. FastText và véc-tơ câu . . . 4

1.3.4. Đối chiếu các biểu diễn . . . 5

1.4. Các mô hình phân loại truyền thống . . . 5

1.4.1. Naive Bayes . . . 5

1.4.2. MultinomialNB và GaussianNB . . . 5

1.4.3. SVM tuyến tính . . . 6

1.5. Mô hình ngôn ngữ tiền huấn luyện và viBERT . . . 6

1.5.1. Transformer encoder và self-attention . . . 6

1.5.2. BERT và học chuyển giao . . . 6

1.5.3. viBERT của FPT.AI . . . 7

1.5.4. WordPiece và đầu vào mô hình . . . 7

1.5.5. Fine-tuning cho phân loại hai lớp . . . 7

1.6. Thiết kế thực nghiệm và xử lý mất cân bằng . . . 8

1.6.1. Train, validation, test và rò rỉ dữ liệu . . . 8

1.6.2. CV, OOF và lựa chọn siêu tham số . . . 8

1.6.3. Mất cân bằng, trọng số và giảm mẫu . . . 8

1.6.4. Điều chỉnh ngưỡng quyết định . . . 9

1.7. Đánh giá và phân tích mô hình . . . 9

1.7.1. Ma trận nhầm lẫn và chỉ số nhị phân . . . 9

1.7.2. Đường Precision–Recall và Average Precision . . . 9
1.7.3. Bootstrap và phân tích lỗi . . . 10

CHƯƠNG 2. THỰC NGHIỆM VÀ KẾT QUẢ 11

2.1. Thu thập và chuẩn bị dữ liệu . . . 11
2.1.1. Quy trình và môi trường thực nghiệm . . . 11
2.1.2. Nguồn và phương pháp thu thập . . . 12
2.1.3. Chuyển đổi và làm sạch . . . 12
2.1.4. Gán nhãn Spam/Ham . . . 13
2.1.5. Dữ liệu thực nghiệm v7 và khảo sát . . . 14
2.1.6. Tiền xử lý đầu vào cho hai nhánh . . . 14
2.1.7. Phân chia tập và kiểm soát rò rỉ . . . 15

2.2. Xây dựng và huấn luyện mô hình truyền thống . . . 16
2.2.1. BoW và TF-IDF . . . 16
2.2.2. FastText . . . 16
2.2.3. Sáu tổ hợp B1–B6 . . . 16
2.2.4. Chọn cách xử lý từ dừng bằng CV . . . 16
2.2.5. Ba chiến lược trên TF-IDF + SVM . . . 17

2.3. Xây dựng và fine-tune viBERT . . . 18
2.3.1. Đầu vào và kiến trúc phân loại . . . 18
2.3.2. Cấu hình huấn luyện . . . 18
2.3.3. So sánh loss trên validation . . . 19
2.3.4. Huấn luyện cuối và lưu checkpoint . . . 20
2.3.5. Tái sử dụng thực nghiệm . . . 20

2.4. Đánh giá và so sánh kết quả . . . 20
2.4.1. Giao thức đánh giá . . . 20
2.4.2. Sáu tổ hợp truyền thống . . . 20
2.4.3. Tác động của ba chiến lược SVM . . . 21
2.4.4. Kết quả viBERT và đối chiếu S3 . . . 23
2.4.5. Phân tích lỗi và độ bất định . . . 25

2.5. Lưu mô hình và ứng dụng bình luận mới . . . 26
2.5.1. Sản phẩm lưu và cách nạp lại . . . 26
2.5.2. Luồng suy luận . . . 26
2.5.3. Minh họa dự đoán mới . . . 26

2.6. Thảo luận và nhận xét thực nghiệm . . . 27
2.6.1. Những kết quả đạt được . . . 27

2.6.2. Đánh đổi và chi phí 27
2.6.3. Giới hạn của nghiên cứu 27

KẾT LUẬN **28**

TÀI LIỆU THAM KHẢO **29**

DANH MỤC BẢNG

Bảng 1.1. Đặc điểm ba phương pháp biểu diễn văn bản 5
Bảng 2.1. Năm nhóm Facebook trong snapshot thu thập 12
Bảng 2.2. Số lượng qua các bước xử lý ở Notebook 01 13
Bảng 2.3. Phân bố nhãn trong CSV của Notebook 01 13
Bảng 2.4. Thành phần và nhãn của bộ dữ liệu v7 14
Bảng 2.5. Ví dụ tiền xử lý cùng ID trên v7 15
Bảng 2.6. Phân chia dữ liệu cho hai nhánh mô hình 15
Bảng 2.7. Sáu tổ hợp mô hình truyền thống cơ sở 16
Bảng 2.8. F1 spam CV khi giữ và loại từ dừng 17
Bảng 2.9. Các chiến lược TF-IDF + SVM trên train/CV 17
Bảng 2.10. Cấu hình fine-tuning viBERT 18
Bảng 2.11. Kết quả hai loss viBERT trên validation 19
Bảng 2.12. Kết quả test của sáu mô hình cơ sở 20
Bảng 2.13. Kết quả test của B4 và ba chiến lược SVM 22
Bảng 2.14. Đối chiếu S3 và M1 trên cùng tập test 23
Bảng 2.15. Một số trường hợp dự đoán khó theo ID 25
Bảng 2.16. Minh họa suy luận viBERT trên câu mới 26

DANH MỤC HÌNH

Hình 2.1. Luồng dữ liệu và hai nhánh phân loại của project 11
Hình 2.2. Phân bố nhãn ham/spam trong v7 14
Hình 2.3. Số mẫu và trọng số lớp ở các chiến lược SVM 17
Hình 2.4. F1 spam trên validation theo epoch của hai loss viBERT 19
Hình 2.5. Ma trận nhầm lẫn của sáu mô hình cơ sở 21
Hình 2.6. Ma trận nhầm lẫn B4 và ba chiến lược TF-IDF + SVM 22
Hình 2.7. Ma trận nhầm lẫn của viBERT trên tập test chung 24
Hình 2.8. Đường Precision–Recall của S3 và M1 trên cùng 7.474 mẫu test 25

TỪ VIẾT TẮT

AP	Average Precision
BoW	Bag-of-Words
CV	Cross-validation
FN/FP	Bỏ sót spam / gắn nhầm ham
M1	Mô hình viBERT được chọn
NB	Naive Bayes
OOF	Out-of-fold
PR	Precision–Recall
S3	TF-IDF + SVM với ngưỡng OOF
SVM	Support Vector Machine
TF-IDF	Term Frequency–Inverse Document Frequency
TP/TN	Dự đoán đúng spam / đúng ham

MỞ ĐẦU

Các nhóm Facebook của sinh viên PTIT là nơi trao đổi lịch học, học phần, tài liệu và kinh nghiệm. Cùng với các trao đổi hữu ích, luồng bình luận có thể chứa lời chào mời, quảng cáo, dẫn liên kết hoặc nội dung không phù hợp với mục đích thảo luận. Khi số bình luận tăng, việc đọc và sàng lọc thủ công trở nên tốn thời gian. Đề tài xây dựng một bộ phân loại hỗ trợ nhận biết bình luận **ham** (nội dung bình thường) và **spam** (nội dung cần xem xét), dựa trên văn bản bình luận của cộng đồng sinh viên PTIT. Đây là công cụ hỗ trợ phân loại, không thay thế quyết định kiểm duyệt của con người.

Mục tiêu của nhóm là thực hiện một quy trình có thể giải thích từ thu thập, chuyển dữ liệu, làm sạch, gán nhãn, biểu diễn văn bản và huấn luyện đến đánh giá và dự đoán bình luận mới. Phần mô hình truyền thống khảo sát ba biểu diễn Bag-of-Words (BoW), TF-IDF, FastText với hai họ thuật toán Naive Bayes (NB) và máy véc-tơ hỗ trợ tuyến tính (SVM). Phần mở rộng fine-tune viBERT, một mô hình tiếng Việt đã tiền huấn luyện. Hai nhánh được so sánh trên cùng tập kiểm tra và cùng quy ước spam là lớp dương.

Quy trình thu thập và gán nhãn được trình bày bằng bình luận thực tế đã lưu từ các nhóm PTIT. Đợt dữ liệu tái hiện trong Notebook 01 cho ra 6.289 bình luận có nhãn. Phần huấn luyện sử dụng bản dữ liệu v7 đã chốt gồm 38.233 bình luận, với 32.907 ham và 5.326 spam. Hai phạm vi này được nêu riêng; một lần chạy Notebook 01 không tự tạo ra toàn bộ v7. Dữ liệu v7 gồm văn bản gốc, văn bản đã làm sạch, nhãn và một số trường mô tả; thực nghiệm phân loại chỉ dùng tín hiệu văn bản, không dùng ảnh, tác giả hay quan hệ giữa các bình luận.

Bài báo cáo gồm hai chương. Chương 1 trình bày những khái niệm cần để hiểu và kiểm tra phương pháp. Chương 2 mô tả quy trình dữ liệu, hai nhánh huấn luyện, kết quả trên tập kiểm tra, cách lưu mô hình và minh họa sử dụng. Phần cuối tổng kết các kết quả và giới hạn của thực nghiệm.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

1.1. Bài toán phân loại văn bản

1.1.1. Học có giám sát và phân loại nhị phân

Trong học có giám sát, mỗi mẫu huấn luyện gồm đầu vào x và nhãn y đã biết. Mô hình học một quy tắc f(x) để dự đoán nhãn cho văn bản mới. Với bài toán này, x là nội dung của một comment hoặc reply; y thuộc {ham, spam}. Nhóm mã hóa ham = 0, spam = 1 và xem spam là lớp dương. Cách quy ước này quyết định ý nghĩa của Precision, Recall và F1 trong toàn báo cáo. Việc dự đoán một nhãn từ văn bản khác với việc xác định chắc chắn ý định người viết: những câu ngắn hoặc không có ngữ cảnh có thể không đủ thông tin để suy đoán mục đích [1].

1.1.2. Đặc điểm bình luận tiếng Việt

Bình luận mạng xã hội thường ngắn, nhiều chữ viết tắt, tên riêng, số điện thoại, liên kết, emoji và lỗi gõ. Một từ có thể gồm nhiều âm tiết cách nhau bởi dấu trắng; do đó tách theo khoảng trắng chưa phải tách từ tiếng Việt. Nội dung như “ib”, “zl” hoặc một dấu chấm đơn lẻ có thể thiếu ngữ cảnh để phân loại. Đây là những hiện tượng nhóm quan sát trong dữ liệu, không phải một kết luận rằng mọi nhóm sinh viên đều có cùng phân bố ngôn ngữ. Dữ liệu văn bản được xem cùng một đơn vị mẫu là comment/reply; bài viết gốc không được đưa vào đầu vào phân loại.

1.1.3. Gán nhãn có hỗ trợ AI

Gán nhãn tạo cặp (văn bản, ham/spam) để huấn luyện. Một mô hình ngôn ngữ có thể đưa ra nhãn gợi ý và điểm confidence; hệ thống gán nhãn có thể lưu gợi ý để người dùng chấp nhận hoặc duyệt. Gợi ý chưa tự động là nhãn đã thẩm định. Trong thực nghiệm này, mã của nhóm tiếp nhận tự động gợi ý hợp lệ có confidence từ 0,75; giá trị này là quy tắc vận hành, chưa được chứng minh là xác suất đúng đã hiệu chỉnh. Vì vậy, nhiễu nhãn vẫn là rủi ro, đặc biệt với câu thiếu ngữ cảnh. Argilla cung cấp cơ chế quản lý suggestions và responses; việc dùng chế độ xuất offline trong Notebook 01 không đồng nghĩa toàn bộ mẫu đã được duyệt thủ công trên giao diện [2].

1.2. Làm sạch và tiền xử lý văn bản

1.2.1. Chuẩn hóa và bảo toàn dữ liệu

Chuẩn hóa Unicode NFC chuyển các chuỗi tương đương về một dạng biểu diễn nhất quán, hữu ích khi so khớp hoặc khử trùng [3]. Sau đó có thể giải mã HTML entity, loại thẻ HTML, thu gọn khoảng trắng và kiểm tra chuỗi rỗng. Khi làm sạch, cần giữ riêng `text` gốc và `cleaned_text` để đối chiếu, đồng thời giữ ID và metadata cho việc nối nhãn, phân tích lỗi. Bản ghi trùng theo một khóa có thể được loại để tránh đếm lặp, song khóa khử trùng phải được định nghĩa rõ. Làm sạch đầu vào gán nhãn và tiền xử lý đầu vào mô hình là hai mục đích khác nhau: lần đầu tạo bảng văn bản dùng để ghi nhãn; lần sau tạo đặc trưng theo yêu cầu từng mô hình.

1.2.2. Tách từ và từ dừng

Tách từ tiếng Việt xác định ranh giới từ trong chuỗi gồm nhiều âm tiết. Chẳng hạn, một biểu diễn có thể ghép "cải thiện" thành `cải_thiện`, trong khi tách theo dấu trắng sẽ coi đó là hai đơn vị. Project dùng `underthesea.word_tokenize(..., format="text")` cho nhánh truyền thống [4]. Từ dừng là các từ thường gặp có thể mang ít thông tin cho một nhiệm vụ; tuy nhiên một từ chức năng vẫn có thể góp phần phân biệt câu hỏi sinh viên với câu rao. Việc giữ hay loại từ dừng được chọn bằng đánh giá chéo trên train, không mặc định loại bỏ.

1.2.3. Tín hiệu đặc biệt và giới hạn che tên

Đường dẫn và số điện thoại có thể là tín hiệu hữu ích cho spam. Thay chúng bằng `urltoken` và `phonetoken` giúp giữ sự hiện diện của tín hiệu mà giảm phụ thuộc vào địa chỉ hoặc số cụ thể. Emoji có thể được gom thành `emoji_token` ở nhánh truyền thống. Hàm xử lý còn che một số tên/username ở đầu câu theo quy tắc bảo thủ. Những phép biến đổi này không bảo đảm ẩn danh hoàn toàn và có thể làm mất manh mối cần thiết trong một số trường hợp. Nhánh viBERT giữ phần lớn văn bản gốc, chỉ chuẩn hóa Unicode, HTML và khoảng trắng để không phá cấu trúc đầu vào mà checkpoint đã học.

1.3. Các phương pháp biểu diễn văn bản

1.3.1. Bag-of-Words

BoW xây dựng từ vựng trên tập huấn luyện, rồi biểu diễn mỗi văn bản bằng số lần xuất hiện của từng đơn vị từ/ngram. Nếu từ vựng có V mục, mỗi văn bản trở thành một hàng của ma trận N × V, thường rất thưa. Unigram mô tả một đơn vị; bigram giữ được một phần thông tin thứ tự cục bộ. BoW dễ giải thích và hợp với dữ liệu văn bản ngắn nhưng không trực tiếp nắm ngữ cảnh xa hoặc sự tương đồng giữa các từ đồng nghĩa [1], [5]. Trong project, `CountVectorizer` được fit trong từng fold, `min_df=2` và khảo sát n-gram (1,1) hoặc (1,2).

1.3.2. TF-IDF

TF-IDF (term frequency–inverse document frequency) tăng trọng số cho một đơn vị xuất hiện trong văn bản đang xét nhưng ít phổ biến trong toàn bộ tập học. Với `TfidfVectorizer` của project, tần suất dưới tuyến tính và IDF làm trơn được tính trước khi chuẩn hóa L2. Công thức dưới đây phản ánh cấu hình đang dùng, với tf(t,d) > 0, N là số văn bản train của fold và df(t) là số văn bản chứa t [5].

$$tf'(t,d) = 1 + \log(tf(t,d)); \quad idf(t) = \log((1 + N)/(1 + df(t))) + 1; \quad v(d) = tf' \times idf / |tf' \times idf||_2| \quad (1.1)$$

TF-IDF vẫn tạo ma trận thưa và không tự hiểu nghĩa theo ngữ cảnh. So với BoW, một từ xuất hiện ở hầu hết bình luận có trọng số thấp hơn; điều này có thể giúp phân loại nhưng hiệu quả phụ thuộc dữ liệu và thuật toán ghép phía sau. Vocabulary và IDF chỉ được học trên train của mỗi fold, rồi mới transform validation/test.

1.3.3. FastText và véc-tơ câu

FastText biểu diễn từ bằng các n-gram ký tự, cho phép chia sẻ thông tin giữa những dạng viết có chung các mảnh ký tự. Bài báo gốc mở rộng skip-gram theo hướng này và cho phép tạo véc-tơ cho từ ngoài từ vựng nhờ subword [6]. Trong project, embedding học **trên tập train của từng fold**, không tải sẵn embedding ngoài: skip-gram, 100 chiều, n-gram ký tự độ dài 3–5, `min_count=2`, 10 epoch. Đây là véc-tơ từ; để phân loại một bình luận, nhóm lấy trung bình véc-tơ từ hoặc trung bình có trọng số TF-IDF. Pooling câu là lựa chọn triển khai của nhóm, không phải kết quả có sẵn từ bài báo.

$$v(\text{câu}) = [\Sigma_t a_t \cdot e_t] / [\Sigma_t a_t], \text{ với } a_t = \text{count}(t) \text{ hoặc } \text{count}(t) \times idf(t) \quad (1.2)$$

Véc-tơ câu dày và gọn hơn BoW/TF-IDF, nhưng phép trung bình làm mất nhiều thông tin về vị trí và sự phủ định. Subword của FastText có thể giúp từ hiếm, song không đảm bảo phân biệt được một câu rao với câu hỏi cùng chứa từ “link”. Trong sáu

mô hình cơ sở, FastText được ghép với GaussianNB hoặc với StandardScaler + LinearSVC.

1.3.4. Đối chiếu các biểu diễn

Bảng 1.1. Đặc điểm ba phương pháp biểu diễn văn bản

Biểu diễn	Kiểu véc-tơ	Thông tin giữ lại	Giới hạn chính
BoW	Thưa, chiều theo từ vựng	Tần suất từ/ngram	Không có nghĩa ngữ cảnh; dễ lệch theo từ phổ biến
TF-IDF	Thưa, chiều theo từ vựng	Từ/ngram và mức hiếm trong corpus train	Không trực tiếp hiểu nghĩa hoặc thứ tự xa
FastText + pooling	Dày, 100 chiều	Mảnh ký tự và trung bình embedding	Pooling làm mất cấu trúc câu

1.4. Các mô hình phân loại truyền thống

1.4.1. Naive Bayes

Theo định lý Bayes, P(y|x) tỷ lệ với P(x|y)P(y). Naive Bayes (NB) đơn giản hóa P(x|y) bằng giả định các đặc trưng độc lập có điều kiện khi đã biết nhãn. Giả định này hiếm khi đúng hoàn toàn với ngôn ngữ, nhưng cho phép ước lượng nhanh trên dữ liệu nhiều chiều và có thể làm đường cơ sở rõ ràng [1], [7]. Quy tắc dự đoán chọn lớp có hậu nghiệm lớn hơn; đối với NB trong báo cáo, ngưỡng spam mặc định là 0,5.

1.4.2. MultinomialNB và GaussianNB

MultinomialNB mô hình hóa các giá trị không âm kiểu đếm hoặc trọng số từ; project dùng biến thể này với BoW và TF-IDF. Tham số alpha làm trơn cộng, tránh xác suất bằng 0 khi một từ chưa gặp trong lớp; chỉ alpha = 1 mới tương ứng cách gọi Laplace thông thường. GaussianNB giả định mỗi chiều véc-tơ liên tục có phân phối Gaussian trong từng lớp, phù hợp về kiểu dữ liệu hơn với embedding FastText; `var_smoothing` bổ sung ổn định số. Giả định Gaussian là giả định mô hình, không tự trở thành sự thật chỉ vì véc-tơ có giá trị âm [7].

$$\hat{y} = \operatorname{argmax}_{\gamma} P(y) \times \prod_{j} P(x_j \, y)| (1.3)$$

1.4.3. SVM tuyến tính

SVM tuyến tính tìm siêu phẳng phân tách các lớp bằng một biên quyết định. Hệ số C điều chỉnh đánh đổi giữa độ rộng biên và lỗi trên train; C lớn làm phạt lỗi mạnh hơn. `LinearSVC` dùng thư viện LIBLINEAR và trong cấu hình này dùng L2 cùng `squared_hinge`, `dual=False` [8], [9]. Điểm `decision_function` cho biết mẫu nằm về phía nào của biên, **không phải xác suất spam**. Ngưỡng mặc định 0 chọn phía spam khi score > 0; ngưỡng âm chọn spam cho cả một số mẫu trước đây nằm phía ham. Véc-tơ FastText được StandardScaler trước SVM; BoW/TF-IDF không cần bước này trong pipeline hiện hành.

$$L(w,b) \approx (1/2)\,|w||^2 + C \cdot \Sigma_i \max(0,\, 1 - y_i(w \cdot x_i + b))^2, \text{ với } y_i \in \{-1,+1\} \quad (1.4)$$

Công thức (1.4) mô tả ý tưởng regularization và squared hinge; chi tiết xử lý intercept của LIBLINEAR được xác định bởi triển khai `LinearSVC`, không suy ra trực tiếp từ ký hiệu giản lược này. SVM tuyến tính thuận tiện cho ma trận BoW/TF-IDF thưa có nhiều chiều.

1.5. Mô hình ngôn ngữ tiền huấn luyện và viBERT

1.5.1. Transformer encoder và self-attention

Transformer dùng attention để cho mỗi vị trí kết hợp thông tin từ các vị trí khác trong chuỗi. Một lớp encoder gồm multi-head self-attention và mạng feed-forward, kết hợp residual connection và layer normalization. Với Q (query), K (key), V (value), attention chuẩn hóa độ phù hợp giữa từng query và các key rồi trộn các value [10]. BERT dùng các lớp encoder nên mỗi token có thể nhận thông tin từ cả trái và phải; project không xây một mô hình dịch encoder–decoder.

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})\, V \quad (1.5)$$

1.5.2. BERT và học chuyển giao

BERT là viết tắt của *Bidirectional Encoder Representations from Transformers*. BERT gốc được tiền huấn luyện bằng masked language modeling (MLM) và next sentence prediction (NSP), rồi điều chỉnh trọng số cho tác vụ cụ thể bằng fine-tuning [11]. Việc nhóm làm ở đây là **fine-tune checkpoint có sẵn** cho hai lớp ham/spam, không huấn luyện mô hình ngôn ngữ từ đầu. Biểu diễn ngữ cảnh của BERT khác véc-tơ từ tĩnh FastText: cùng một từ có thể nhận biểu diễn khác tùy câu. Điều đó tạo khả năng xử lý nghĩa theo ngữ cảnh, đổi lại là chi phí huấn luyện lớn hơn.

1.5.3. viBERT của FPT.AI

viBERT là checkpoint BERT tiếng Việt do FPT.AI công bố; nhóm dùng `FPTAI/vibert-base-cased` từ Hugging Face, cố định revision của checkpoint trong cấu hình chạy [12], [13]. Mã repo viBERT dẫn một bài nghiên cứu về gán nhãn chuỗi tiếng Việt, không phải công trình báo cáo kết quả spam PTIT [12]. Vì model card của checkpoint không mô tả đầy đủ mọi nguồn tiền huấn luyện, báo cáo không suy đoán quy mô corpus hay giấy phép từ tên mô hình. Kiến trúc phân loại của nhóm nạp encoder từ checkpoint, thêm head hai lớp và fine-tune encoder cùng head trên dữ liệu v7.

1.5.4. WordPiece và đầu vào mô hình

Tokenizer BERT phân nhỏ từ thành các mảnh WordPiece; tiền tố `##` chỉ một mảnh nối tiếp từ trước. `[CLS]` được thêm ở đầu, `[SEP]` kết thúc chuỗi, `[PAD]` lấp đầy batch và `[UNK]` thay mảnh không biểu diễn được trong từ vựng [11], [14]. Tokenizer đổi văn bản thành `input_ids` và `attention_mask`; đầu vào quá dài bị cắt ở 128 token trong thực nghiệm. Dùng tokenizer `cased` nên giữ phân biệt chữ hoa/thường. `[UNK]` ở câu có ký tự ngoại ngữ không phải lỗi font hiển thị; đó là kết quả của tokenizer trên nội dung thực.

1.5.5. Fine-tuning cho phân loại hai lớp

Head phân loại biến biểu diễn của câu thành hai logits. Softmax chuẩn hóa logits thành xác suất dự đoán hai lớp; cross-entropy phạt xác suất thấp ở lớp đúng. Khi dùng loss có trọng số, lỗi của một lớp được nhân với hệ số của lớp đó [15]. Nhóm thử loss thường và loss có trọng số từ số mẫu của tập fit; đây là hai phương án phát triển, chỉ một phương án được chọn cho huấn luyện cuối. AdamW tách weight decay khỏi bước cập nhật thích ứng [16]. Learning rate, warmup, batch, gradient accumulation và FP16 là các thành phần cấu hình huấn luyện được ghi lại trong kết quả, không phải những phương pháp phân loại độc lập [17].

$$p(y=k\ x) = \exp(z_k) / \Sigma_j \exp(z_j);\ CE = -w_\gamma \log p(y \mid x) \quad (1.6)$$

1.6. Thiết kế thực nghiệm và xử lý mất cân bằng

1.6.1. Train, validation, test và rò rỉ dữ liệu

Train dùng để học tham số; validation hoặc các fold CV dùng để chọn cấu hình; test được giữ riêng để báo cáo kết quả sau cùng. Nếu học từ vựng, IDF, scaler hoặc embedding bằng cả test trước khi chia, phép đo trên test sẽ bị rò rỉ. `Pipeline` gói biến đổi và bộ phân loại để fit lại đúng trên train của mỗi fold [18]. Nhóm tạo khóa văn bản từ `cleaned_text` đã chuẩn hóa và gán các khóa giống nhau vào cùng tập/fold. Cách này giảm rò rỉ do văn bản trùng hoặc gần trùng theo khóa, nhưng không bảo đảm tách theo tác giả, nhóm Facebook hay thời gian.

1.6.2. CV, OOF và lựa chọn siêu tham số

Cross-validation (CV) chia train thành nhiều fold: học trên các fold còn lại, dự đoán fold đang giữ, tính chỉ số rồi lấy trung bình. `GridSearchCV` xét lưới tham số cho sáu mô hình truyền thống và chọn theo trung bình F1 spam [18]. Dự đoán out-of-fold (OOF) cho mỗi mẫu train được tạo bởi một mô hình không học mẫu đó. Nhóm dùng các score OOF của SVM để tìm ngưỡng quyết định; đây là bước **lựa chọn trên train**, không phải điểm test hoặc một phép ước lượng nested CV độc lập. viBERT dùng một fold validation trong train để chọn loss/epoch, rồi fit lại toàn train và đánh giá test.

1.6.3. Mất cân bằng, trọng số và giảm mẫu

Khi ham chiếm đa số, mô hình đoán ham cho mọi bình luận vẫn có Accuracy cao nhưng Recall spam bằng 0. Một cách xử lý là tăng trọng số lỗi lớp thiểu số: quy tắc `balanced` đặt trọng số lớp c bằng N/(K·N_c), với K là số lớp [19]. Cách thứ hai là giảm ngẫu nhiên số ham trong phần train cho tỷ lệ ham:spam = 2:1, giữ mọi spam; chỉ áp dụng trong train từng fold, không làm biến đổi test. Giảm mẫu có thể cải thiện khả năng tìm spam nhưng đánh mất một phần ví dụ ham [20]. viBERT cũng thử loss có trọng số tương tự, song việc chọn loss được thực hiện riêng theo validation.

$$w_c = N / (K \cdot N_c); \text{ trong bài } K = 2 \text{ và } c \in \{\text{ham}, \text{spam}\} \quad (1.7)$$

1.6.4. Điều chỉnh ngưỡng quyết định

Cách thứ ba là giữ dữ liệu và trọng số huấn luyện như cũ, nhưng đổi ngưỡng phân loại score. Việc này thay đổi điểm vận hành Precision–Recall, không thay số mẫu hoặc học lại siêu phẳng [21]. Với SVM, score âm vẫn có thể được dự đoán spam nếu vượt một ngưỡng âm hơn. Nhóm dùng các phân vị score OOF, 0 và hai giá trị biên làm ứng viên; chọn ngưỡng có F1 spam trung bình fold cao nhất, nếu hòa thì ưu tiên gần 0. Ba cách trên được thử **riêng** trên TF-IDF + SVM, không phải ba xử lý được áp dụng đồng thời lên cả sáu mô hình.

1.7. Đánh giá và phân tích mô hình

1.7.1. Ma trận nhầm lẫn và chỉ số nhị phân

Với spam là lớp dương, TP là spam được dự đoán spam, FN là spam bị bỏ sót, FP là ham bị đánh dấu spam và TN là ham được dự đoán ham. Accuracy xét mọi dự đoán; Precision trả lời trong các mẫu dự đoán spam có bao nhiêu spam thật; Recall trả lời trong các spam thật tìm được bao nhiêu. F1 là trung bình điều hòa của Precision và Recall [22]. Nếu không dự đoán spam, phần mềm đặt Precision/F1 bằng 0 theo `zero_division=0`, thay vì hiển thị giá trị không xác định.

$$\text{Accuracy}=(TP+TN)/N;\ \text{Precision}=TP/(TP+FP);\ \text{Recall}=TP/(TP+FN);\ F1=2PR/(P+R) \quad (1.8)$$

1.7.2. Đường Precision–Recall và Average Precision

Thay đổi ngưỡng trên một dãy score tạo đường Precision–Recall (PR). Average Precision (AP) tóm tắt thứ tự xếp hạng bằng tổng có trọng số theo độ tăng Recall; trong scikit-learn, AP dùng quy tắc bậc thang, không bằng diện tích hình thang của mọi cách gọi "PR-AUC" [22]. AP của SVM tính từ `decision_function`, của viBERT tính từ xác suất spam; không được hiểu score SVM là xác suất đã hiệu chỉnh. Đường PR và AP đặc biệt hữu ích khi ham đông hơn spam, nhưng vẫn cần xem điểm vận hành cụ thể bằng Precision, Recall và số FP/FN.

$$AP = \Sigma_n (R_n - R_{n-1}) \cdot P_n \quad (1.9)$$

1.7.3. Bootstrap và phân tích lỗi

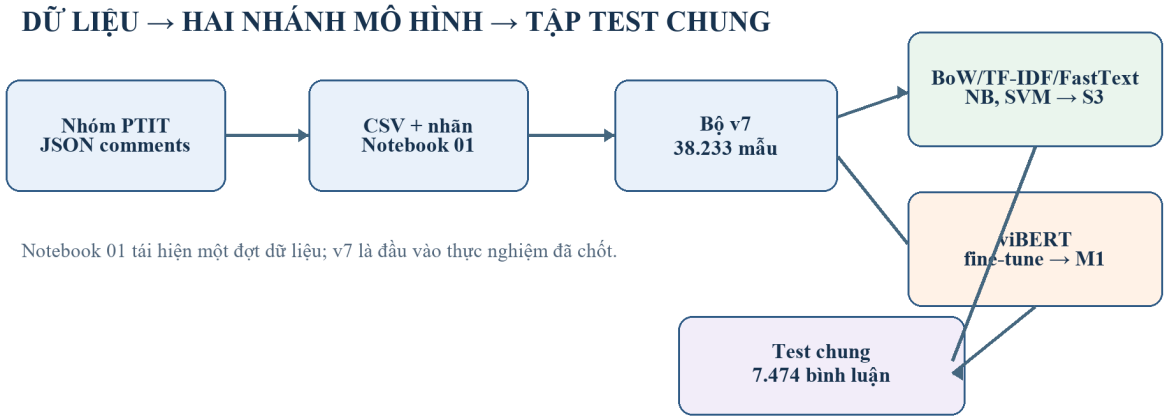
Bootstrap lấy mẫu lại từ tập kiểm tra để mô tả độ dao động của một chỉ số. Project dùng 500 lần lấy mẫu lại theo dòng dự đoán test, lấy phân vị 2,5% và 97,5% của F1 cho từng mô hình truyền thống [23]. Khoảng đó mô tả độ dao động theo cách lấy mẫu đã chọn; nó không chứng minh hai mô hình khác biệt có ý nghĩa thống kê và không được tính cho viBERT trong project. Phân tích lỗi xem trực tiếp FP/FN cùng văn bản và score để hiểu những trường hợp thiếu ngữ cảnh, tên riêng, dấu câu hoặc nhãn khó. Ví dụ lỗi chỉ là quan sát, không đại diện cho toàn bộ dữ liệu nếu chưa thống kê tỷ lệ theo nhóm lỗi.

CHƯƠNG 2. THỰC NGHIỆM VÀ KẾT QUẢ

2.1. Thu thập và chuẩn bị dữ liệu

2.1.1. Quy trình và môi trường thực nghiệm

Quy trình gồm hai chặng dữ liệu và hai nhánh mô hình. Chặng thứ nhất đọc bài viết và bình luận từ các nhóm Facebook PTIT, chuyển JSON lồng nhau thành bảng, làm sạch và tiếp nhận nhãn. Chặng thứ hai dùng bộ dữ liệu v7 đã chốt để huấn luyện mô hình truyền thống hoặc viBERT. Các dự đoán cuối được đối chiếu trên **cùng 7.474 ID test**. Sơ đồ Hình 2.1 chỉ quan hệ giữa các đầu ra, không hàm ý một lần chạy thu thập tái tạo đầy đủ v7.



Hình 2.1. Luồng dữ liệu và hai nhánh phân loại của project

Notebook 01 ghi lại quy trình thu thập và nhãn; Notebook 02 thực hiện tiền xử lý, CV, huấn luyện, đánh giá và demo mô hình truyền thống; Notebook 03 ghi cấu hình huấn luyện viBERT, đọc lại lịch sử GPU và chạy đánh giá/demo. Mã chức năng được tách vào `pipeline/collect`, `prepare`, `label`, `features`, `modeling`, `modern` và `inference`. Nhánh truyền thống được chạy trên Python 3.12 local; nhánh viBERT được fine-tune trên Google Colab với NVIDIA A100-SXM4-80GB, Python 3.13.16, PyTorch 2.11.0+cu130, Transformers 4.57.6 và scikit-learn 1.6.1. Số liệu môi trường lấy từ `run_config.json` của lần huấn luyện.

2.1.2. Nguồn và phương pháp thu thập

Runner `pipeline/collect/scrape_ptit_groups.py` khai báo các nhóm PTIT và điều phối bộ mã crawler được bàn giao trong `external/facebook_scraper/`. Mã nền có nguồn tham khảo công khai từ repo `facebook_post_comment_scraper` [24]; phần runner theo nhóm PTIT là phần cấu hình/điều phối của project. Quy trình nhận các trang bài viết, lấy comment và reply của từng bài, ghi JSON/JSONL và trạng thái khi cần. JSON được kiểm tra trong Notebook 01 chứa **2.170 bài thuộc 5 nhóm** sau; con số này là phạm vi snapshot đã lưu, không phải cam kết đã thu toàn bộ bài của mọi nhóm trong runner.

Bảng 2.1. Năm nhóm Facebook trong snapshot thu thập

Nhóm trong snapshot	ID nhóm	Số bài
D26 (2008) PTIT – Học viện Công nghệ Bưu chính Viễn thông	1130813927934700	500
PTIT Confessions – Sinh viên Học viện Công nghệ Bưu chính Viễn thông	1210825580901941	500
D26 Học viện Công nghệ Bưu chính Viễn thông (2K8 PTIT)	1605563914144178	500
Cộng đồng sinh viên PTIT	450310163844833	500
D21 PTIT Hà Nội	778910719727961	170

Mỗi post chứa các trường như `post_id`, `group_id`, `group_name`, thời gian, danh sách comments; mỗi comment có thể chứa replies. `pipeline/collect/json_to_csv.py:flatten_posts` biến comment và reply thành từng hàng. Thứ tự post → comment → reply chỉ mô tả cấu trúc thu thập; nhãn ham/spam chưa có ở bước crawler. Trong Notebook 01, nhóm đọc snapshot JSON đã lưu thay vì gửi yêu cầu Facebook mới. Vì vậy số liệu xuất lại trong báo cáo là số liệu tái hiện từ tệp lưu, không phải một lần crawl trực tiếp trong ngày viết báo cáo.

2.1.3. Chuyển đổi và làm sạch

Sau khi làm phẳng JSON, bảng CSV chưa nhãn gồm `post_id`, `group_id`, `group_name`, `created_at`, `comment_type`, `text`, `reaction_count`. Dòng không có text bị bỏ. Hàm `clean_facebook_comments` giữ bản gốc ở `text`, thêm `cleaned_text`, khử trùng theo `(post_id, comment_type, text)` và áp dụng chuẩn hóa Unicode/HTML, URL/số điện thoại, emoji, dấu câu, khoảng trắng. `urltoken` và `phonetoken` vẫn giữ thông tin có liên kết hoặc số điện thoại. ID được nối theo bản ghi đã lưu trước khi gán nhãn để tránh gắn gợi ý sang bình luận khác.

Bảng 2.2. Số lượng qua các bước xử lý ở Notebook 01

Mốc trong Notebook 01	Số lượng	Ý nghĩa
Bài viết JSON	2.170	Post nằm trong snapshot của 5 nhóm
Comment và reply đã làm phẳng	13.833	Hàng văn bản trước lọc rỗng
Hàng có text	13.704	Bỏ 129 hàng text rỗng
Hàng sau làm sạch/khử trùng	10.974	Bảng `comments_cleaned.csv` dùng cho nhãn
Hàng nối được với phản hồi đã lưu	6.571	Có thể xem nhãn gợi ý theo ID
Hàng được tiếp nhận nhãn	6.289	CSV cuối của Notebook 01

Chênh lệch 13.704 → 10.974 bao gồm các điều kiện làm sạch/khử trùng; không được coi toàn bộ 2.730 hàng đó chỉ là bản ghi trùng. Tương tự, 6.571 → 6.289 do quy tắc tiếp nhận/nhãn hợp lệ, không nên suy đoán từng lý do nếu chưa thống kê sâu. Ví dụ thực tế trong CSV: `cmt_100` của v7 có số điện thoại được thay thành `phonetoken`; việc thay token giữ tín hiệu liên hệ mà không dùng chính số cụ thể làm đặc trưng.

2.1.4. Gán nhãn Spam/Ham

Mã trong `pipeline/label/ai_assisted_spam_ham.py` hỗ trợ tạo gợi ý bằng GLM qua OpenRouter, có cache để tránh tính lại; `upload_to_argilla.py` và `download_labeled_data.py` hỗ trợ quy trình dùng giao diện duyệt nhãn [2]. Trong luồng tái hiện ở Notebook 01, phản hồi đã lưu được nối vào bảng sạch bằng `cached_annotations.attach_annotation_ids`, kiểm tra khóa và văn bản, rồi `auto_accept_labels.py --skip-argilla` tiếp nhận những gợi ý có nhãn ham/spam và confidence ít nhất **0,75**. Kết quả có `official_label` được xuất thành `data/labeled/comments_labeled.csv`.

Bảng 2.3. Phân bố nhãn trong CSV của Notebook 01

Nhãn trong CSV của Notebook 01	Số hàng	Tỷ lệ
ham	5.190	82,53%
spam	1.099	17,47%
Tổng	6.289	100%

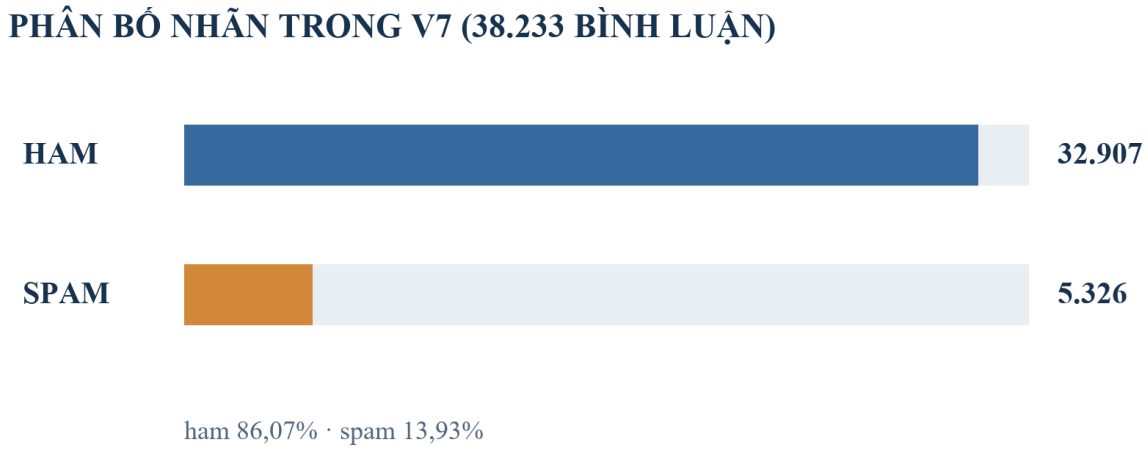
Notebook 01 không gọi API AI hoặc Argilla khi chạy lại với các tệp đã lưu. Nhãn được tiếp nhận tự động theo ngưỡng không có nghĩa đã có người rà thủ công toàn bộ. Ngưỡng confidence 0,75 dành cho **nhãn dữ liệu**, khác ngưỡng score −0,1776 của SVM và ngưỡng xác suất 0,5 của viBERT ở giai đoạn dự đoán.

2.1.5. Dữ liệu thực nghiệm v7 và khảo sát

Đầu vào huấn luyện là `data/final/labeled_v7.csv`, bản dữ liệu v7 đã chốt của nhóm và được giới thiệu tại địa chỉ dataset [25]. Nhóm kiểm trực tiếp số liệu từ CSV local; báo cáo không suy ra toàn bộ lịch sử tạo v7 từ một lần chạy Notebook 01. Bảng gồm 38.233 ID duy nhất, các cột `id`, `text`, `cleaned_text`, `label`, `comment_type`, `reaction_count`. Trong đó 23.388 dòng là comment và 14.845 dòng là reply.

Bảng 2.4. Thành phần và nhãn của bộ dữ liệu v7

Chỉ tiêu v7	ham	spam	Tổng
Số bình luận	32.907	5.326	38.233
Tỷ lệ	86,07%	13,93%	100%



Hình 2.2. Phân bố nhãn ham/spam trong v7

Lớp ham gấp khoảng 6,18 lần spam. Một bộ phân loại luôn đoán ham sẽ có Accuracy khoảng 86,07% trên toàn v7 nhưng Recall spam bằng 0; do đó Accuracy riêng lẻ không đủ. Bình luận trong v7 có nhiều kiểu: câu hỏi học tập như `cmt_0`, lời liên hệ kèm số điện thoại như `cmt_100`, câu có liên kết và cả câu chỉ có dấu câu. Một số nhãn ở câu quá ngắn khó kiểm chứng từ văn bản đơn độc, là giới hạn cần nêu khi đọc kết quả mô hình.

2.1.6. Tiền xử lý đầu vào cho hai nhánh

Nhánh truyền thống đọc `cleaned_text`, áp dụng `pipeline/prepare/preprocess_for_model.py` và lưu kết quả cho toàn bộ v7 ở `data/processed/v7_preprocessed.csv`. Hàm che một số tên/username theo quy tắc, chuẩn hóa URL/số/emoji, viết thường, rút ký tự lặp và gọi underthesea. Hai cột token giữ và bỏ từ dừng được tạo để so sánh bằng CV. Ví dụ từ đúng CSV đã lưu:

Bảng 2.5. Ví dụ tiền xử lý cùng ID trên v7

ID	Văn bản gốc	Giữ từ dừng	Bỏ từ dừng
cmt_0	Cải thiện nhận bằng có chậm không b nhỉ	cải_thiện nhận bằng có chậm không b nhỉ	cải_thiện nhận bằng chậm không b
cmt_1	Người tham gia ẩn danh 124 k liên quan…	người tham_gia ẩn_danh 124 k liên_quan…	người tham_gia ẩn_danh 124 k liên_quan… (bỏ một số từ chức năng)

Nhánh viBERT đi từ `text` gốc, chuẩn hóa NFC, giải mã HTML và gom khoảng trắng, sau đó dùng tokenizer cased của checkpoint. Không lấy chuỗi token underthesea của SVM đưa thẳng vào viBERT. Hai nhánh khác nhau ở cách biểu diễn văn bản, nhưng dùng cùng nhãn và tập test.

2.1.7. Phân chia tập và kiểm soát rò rỉ

Từ `cleaned_text`, hàm `norm_key` chuẩn hóa chữ thường, số và dấu câu thành khóa văn bản; `bucket(key, "test")` đưa các khóa vào năm nhóm xác định. Nhóm bucket 0 được giữ làm test, bốn nhóm còn lại thành train. Do dựa trên hàm băm khóa, tỷ lệ thực tế không đúng chính xác 80/20 và không phải chia tầng ngẫu nhiên theo nhãn. Các khóa giống nhau không giao nhau giữa train/test. Trên train, `grouped_folds` tạo năm fold theo khóa cho CV. viBERT dùng fold 0 của train làm validation, sau chọn cấu hình mới dùng toàn bộ train để fit cuối.

Bảng 2.6. Phân chia dữ liệu cho hai nhánh mô hình

Tập	ham	spam	Tổng	Vai trò
Train chung	26.384	4.375	30.759	CV truyền thống; nguồn fit/validation viBERT
Fit viBERT	21.204	3.611	24.815	Học khi chọn loss và epoch
Validation viBERT	5.180	764	5.944	Chọn loss/epoch, nằm trong train chung
Test chung	6.523	951	7.474	Đánh giá một lần sau lựa chọn

Trong CV, từ vựng/IDF/FastText/scaler được fit trên phần train của fold. Việc chuẩn hóa theo quy tắc đã định không học thống kê từ test. Tập test chung cho phép so sánh trực tiếp S3 và M1 theo cùng ID, dù CV năm fold của truyền thống và một validation fold của viBERT không phải hai ước lượng cùng giao thức. Cách chia theo text không thay cho đánh giá độc lập theo thời gian hoặc theo nhóm Facebook.

2.2. Xây dựng và huấn luyện mô hình truyền thống

2.2.1. BoW và TF-IDF

`pipeline/modeling/experiments.py` dựng `Pipeline(vec, clf)` để học từ vựng trong fold. BoW dùng `CountVectorizer`, TF-IDF dùng `TfidfVectorizer(sublinear_tf=True)`; cả hai dùng `min_df=2`, token pattern của project và lưới n-gram `(1,1)` hoặc `(1,2)`. Với NB, lưới alpha là `{0,1; 0,3; 1,0}`. Với SVM, lưới C là `{0,1; 0,3; 1,0; 3,0}` và `class_weight=None` cho các mô hình cơ sở. `GridSearchCV` lấy trung bình Precision, Recall, F1 spam trên năm fold và refit theo F1.

2.2.2. FastText

`pipeline/features/spam_features.py:FastTextVectorizer` học embedding skip-gram trong từng fold với kích thước 100, cửa sổ 5, n-gram ký tự 3–5, seed 42 và một worker. Lưới chọn pooling `mean` hoặc `tfidf`. FastText + GaussianNB tìm `var_smoothing` trong $\{10^{-9}; 10^{-7}; 10^{-5}; 10^{-3}\}$; FastText + SVM thêm `StandardScaler` và lưới C `{0,01; 0,1; 1,0}`. Việc học embedding trong fold tốn thời gian hơn biểu diễn đếm từ; không dùng embedding huấn luyện trên cả tập trước CV.

2.2.3. Sáu tổ hợp B1–B6

Sáu mã B1–B6 là **ba biểu diễn × hai họ thuật toán**, không phải sáu bộ dữ liệu. NB cụ thể là MultinomialNB cho BoW/TF-IDF và GaussianNB cho FastText; SVM cụ thể là LinearSVC. Ở mốc cơ sở, chưa dùng trọng số lớp, giảm mẫu hay đổi ngưỡng.

Bảng 2.7. Sáu tổ hợp mô hình truyền thống cơ sở

Mã	Biểu diễn	Bộ phân loại	Điểm dự đoán mặc định
B1	BoW	MultinomialNB	Hậu nghiệm spam ≥ 0,5
B2	BoW	LinearSVC	Decision score > 0
B3	TF-IDF	MultinomialNB	Hậu nghiệm spam ≥ 0,5
B4	TF-IDF	LinearSVC	Decision score > 0
B5	FastText	GaussianNB	Hậu nghiệm spam ≥ 0,5
B6	FastText	StandardScaler + LinearSVC	Decision score > 0

2.2.4. Chọn cách xử lý từ dừng bằng CV

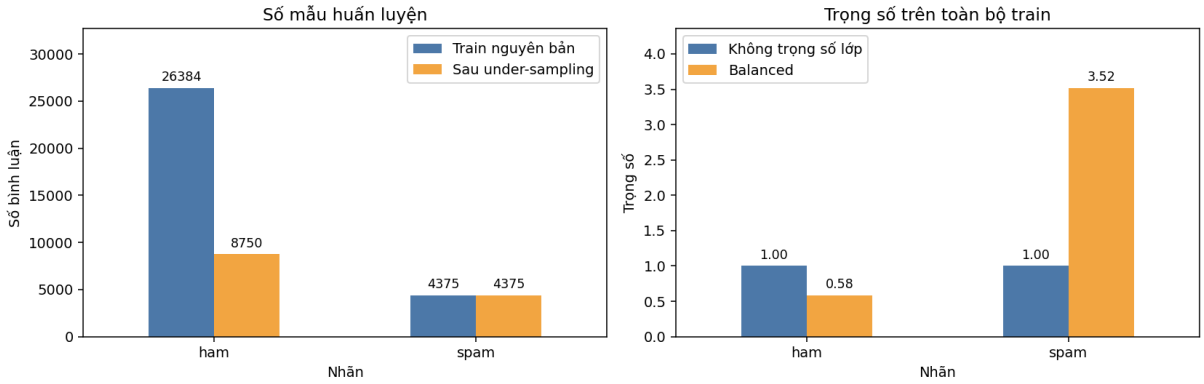
Notebook 02 tính CV F1 spam cho từng cặp biểu diễn–thuật toán ở hai chuỗi token đã chuẩn bị. Cả sáu cặp đều có CV F1 cao hơn khi giữ từ dừng trong lần chạy này. Nhóm do đó tiếp tục với chuỗi **giữ từ dừng**; test không được dùng để chọn nhánh tiền xử lý. Kết quả này chỉ đúng trên dữ liệu/cấu hình đã khảo sát, không có nghĩa loại từ dừng luôn có hại trong mọi tác vụ.

Bảng 2.8. F1 spam CV khi giữ và loại từ dừng

Mã	Loại từ dừng: CV F1	Giữ từ dừng: CV F1
B1	0,5859	0,6077
B2	0,6306	0,7091
B3	0,5835	0,5959
B4	0,7562	0,7736
B5	0,4894	0,5594
B6	0,6653	0,6794

2.2.5. Ba chiến lược trên TF-IDF + SVM

B4 có CV F1 cao nhất trong sáu mô hình cơ sở nên được chọn làm nền để khảo sát mất cân bằng/điểm vận hành. S1 giữ toàn bộ train, đặt `class_weight="balanced"`; S2 lấy toàn bộ spam và lấy ngẫu nhiên ham theo tỷ lệ 2:1 **trong phần train từng fold**; S3 giữ mô hình B4 không trọng số, không giảm mẫu, nhưng chỉnh ngưỡng decision score bằng các dự đoán OOF trên train. Đây là ba thay đổi tách biệt, không ghép tất cả vào một mô hình. Hình 2.3 cho thấy S2 làm giảm số ham trong train cuối từ 26.384 xuống 8.750, còn S1 tăng trọng số lỗi spam lên khoảng 3,52.



Hình 2.3. Số mẫu và trọng số lớp ở các chiến lược SVM

Bảng 2.9. Các chiến lược TF-IDF + SVM trên train/CV

Mã	Chiến lược	Mẫu fit cuối	Ngưỡng score	CV Precision	CV Recall	CV F1
B4	Không điều chỉnh	30.759	0	0,9001	0,6806	0,7736
S1	Balanced	30.759	0	0,7233	0,7964	0,7580
S2	Giảm mẫu ham 2:1	13.125	0	0,8038	0,7276	0,7629
S3	Chỉnh ngưỡng OOF	30.759	−0,1776	0,8490	0,7353	**0,7876**

S3 đứng đầu theo CV F1 của các chiến lược đang xét. Cấu hình cuối của S3 giữ từ dừng, n-gram 1–2, C = 0,3, không trọng số lớp, không lấy mẫu lại; ngưỡng được chọn là **−0,1776279252**. Giá trị âm là ngưỡng của `decision_function`, không phải xác suất âm: một bình luận có score −0,1 lớn hơn ngưỡng này sẽ được báo spam. F1 CV của S3 đã dùng OOF để lựa chọn ngưỡng nên không nên gọi là ước lượng nested CV hoàn toàn độc lập; test được giữ riêng để đánh giá cuối.

2.3. Xây dựng và fine-tune viBERT

2.3.1. Đầu vào và kiến trúc phân loại

`pipeline/modern/data.py` đọc cùng v7 và cùng cách chia test, tạo `model_text` từ văn bản gốc bằng NFC, giải mã HTML và gom khoảng trắng; không xóa stopwords hay đổi về chữ thường. Tokenizer của `FPTAI/vibert-base-cased` sinh ID và attention mask. Mỗi câu tối đa 128 token, dài hơn bị cắt; batch ngắn được padding khi chạy. Checkpoint encoder được nạp từ revision `728a91287e4517d9312066a6aa048fadf4e41e91`; head phân loại hai lớp được khởi tạo mới, sau đó encoder và head cùng được fine-tune.

Trong khảo sát token, bình luận `cmt_10948` có một cụm ký tự Nhật trước phần tiếng Việt; các mảnh không có trong từ vựng hiển thị thành `[UNK]`. Văn bản gốc chứa cụm ký tự đó, không phải lỗi hiển thị UTF-8. Tỷ lệ **0,77%** số câu train vượt 128 token đo mức cắt chuỗi, không phải tỷ lệ `[UNK]`.

2.3.2. Cấu hình huấn luyện

Bảng 2.10. Cấu hình fine-tuning viBERT

Thành phần	Giá trị
Checkpoint	`FPTAI/vibert-base-cased`, revision cố định
Seed; độ dài tối đa	42; 128 token
Epoch tối đa	3
Learning rate; weight decay	2×10^{-5}; 0,01
Warmup ratio	0,1
Batch train; gradient accumulation	8; 2 bước
Batch evaluation; mixed precision	16; FP16 trên GPU
Optimizer; ngưỡng dự đoán	AdamW; xác suất spam ≥ 0,5

`pipeline/modern/train.py` định nghĩa dataset, `ClassificationTrainer.compute_loss` và lệnh huấn luyện. Batch hiệu dụng danh nghĩa là 16 mẫu cho một lần cập nhật trên một GPU nếu không có các điều chỉnh khác của Trainer. Mixed precision giúp thực hiện phép tính phù hợp ở FP16 trên GPU; không có phép benchmark tốc độ hay VRAM công bằng để khẳng định mức tiết kiệm cụ thể.

Một lượt smoke forward/backward chỉ kiểm tra khả năng chạy của checkpoint; số liệu báo cáo là từ lần fine-tune đầy đủ.

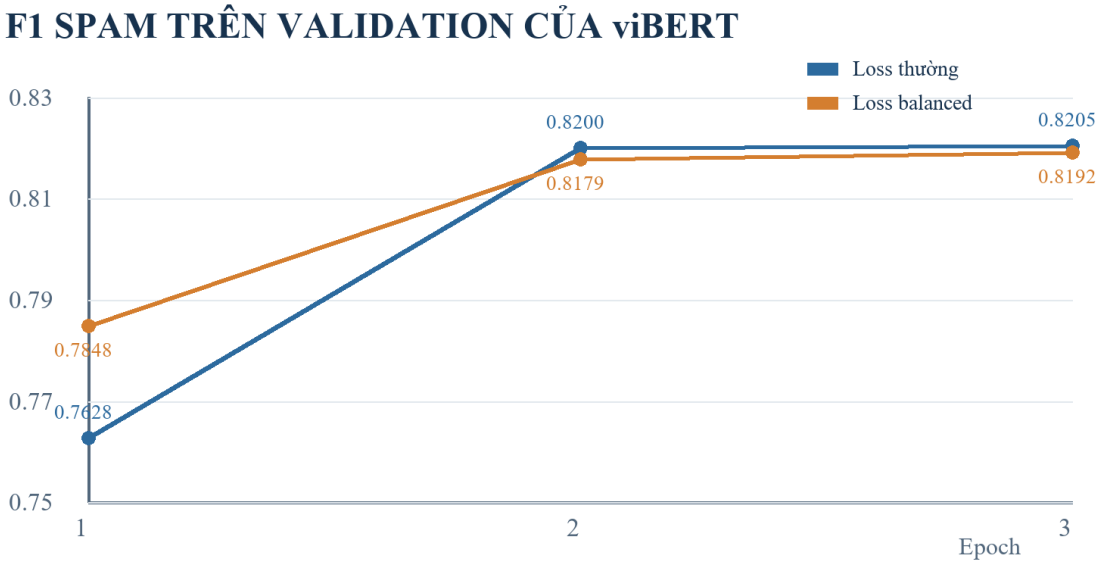
2.3.3. So sánh loss trên validation

Trong giai đoạn phát triển, 24.815 bình luận fit và 5.944 bình luận validation đều nằm trong train chung. Nhóm huấn luyện loss thường (unweighted) và loss balanced tính trọng số từ phân bố nhãn của **fit**, chọn epoch/biến thể theo F1 spam validation. Bảng kết quả dưới đây ghi điểm cao nhất của mỗi biến thể, cả hai ở epoch 3.

Bảng 2.11. Kết quả hai loss viBERT trên validation

Loss	Epoch	Val. Accuracy	Val. Precision	Val. Recall	Val. F1	Val. AP
Thường	3	0,9558	0,8573	0,7866	**0,8205**	0,8819
Balanced	3	0,9549	0,8454	0,7945	0,8192	0,8858

Balanced cho Recall và AP validation nhỉnh hơn; loss thường cho F1 cao hơn khoảng **0,0013** nên được chọn theo tiêu chí đã định. Khoảng cách nhỏ không đủ để khẳng định loss thường luôn ưu việt. Validation F1 ở đây là một fold, không nên đặt cạnh CV F1 năm fold của S3 để tuyên bố viBERT hơn trước khi xem test chung.



Hình 2.4. F1 spam trên validation theo epoch của hai loss viBERT

2.3.4. Huấn luyện cuối và lưu checkpoint

Sau lựa chọn, nhóm nạp lại checkpoint pretrained, khởi tạo head mới và fit mô hình loss thường trong **3 epoch trên toàn bộ 30.759 bình luận train**. Thư mục `artifacts/modern/vibert/model/` lưu `model.safetensors`, tokenizer, config và `inference_config.json` có ngưỡng 0,5. `trainer.save_model` và `tokenizer.save_pretrained` lưu trạng thái đã học; đoạn mã `.py` chỉ mô tả quy trình, không phải bản thân trọng số mô hình. Sau khi lưu, hệ thống nạp lại mô hình từ thư mục local và đối chiếu dự đoán theo ID. Test được đánh giá sau khi loss và epoch đã chốt.

2.3.5. Tái sử dụng thực nghiệm

Notebook 03 có chế độ train đầy đủ trên GPU và chế độ review mặc định để đọc lịch sử/checkpoint đã lưu, kiểm tra và demo mà không fine-tune lại. Chế độ review cho thấy kết quả huấn luyện đã thực hiện; nó không tạo thêm một phép huấn luyện GPU. Checkpoint và các CSV lịch sử là bằng chứng tái sử dụng; để fine-tune mới cần đúng dữ liệu, phiên bản và tài nguyên GPU như cấu hình thực nghiệm.

2.4. Đánh giá và so sánh kết quả

2.4.1. Giao thức đánh giá

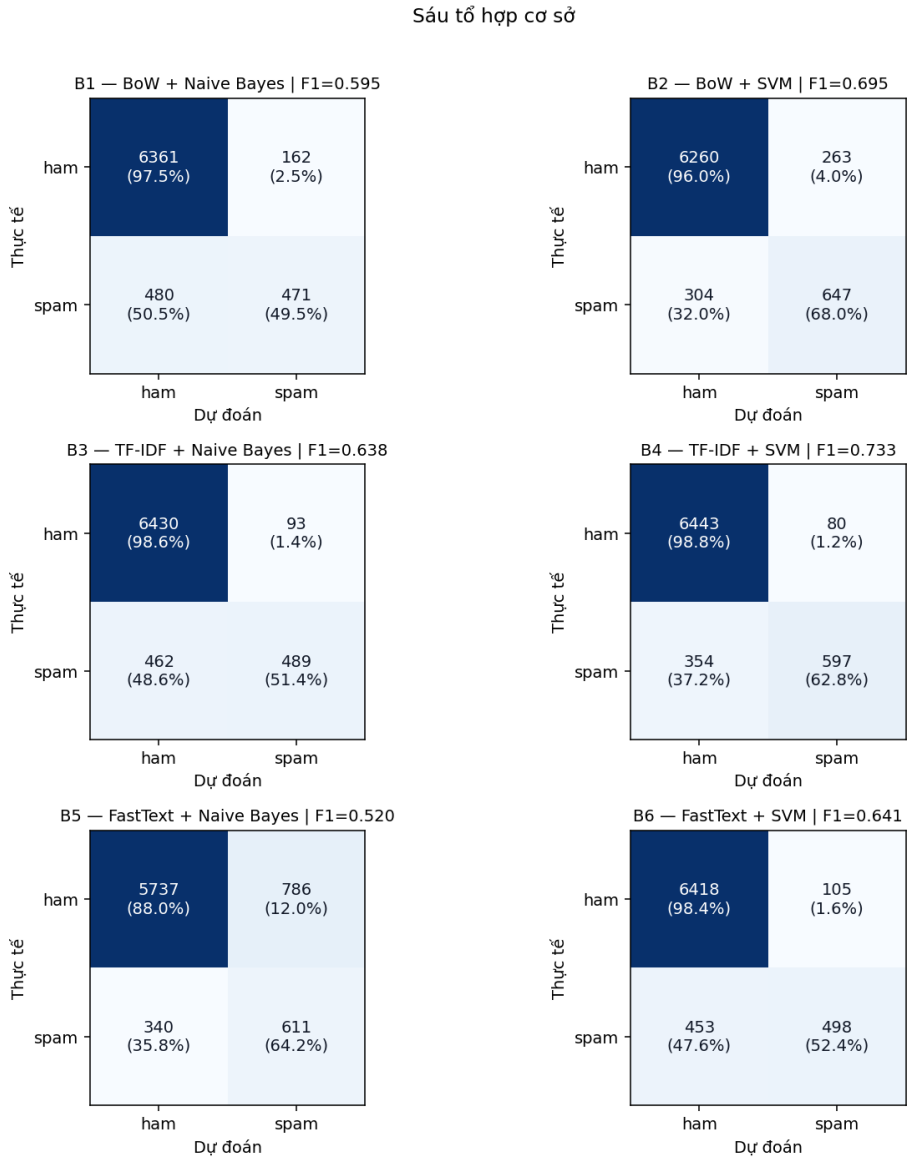
Sau khi chốt mô hình và siêu tham số trên train/CV hoặc validation, nhóm đánh giá trên 7.474 bình luận test gồm 6.523 ham và 951 spam. Các chỉ số báo cáo là Accuracy, Precision, Recall, F1 của lớp spam và Average Precision (AP); ma trận có thứ tự hàng thật/cột dự đoán `[ham, spam]`. S3 và M1 được xác minh có cùng tập ID/nhãn test. Các số in bốn chữ số thập phân để so sánh; CSV kết quả giữ số chính xác hơn. Bảng không dùng điểm validation của viBERT thay điểm test, và không chọn lại ngưỡng theo test.

2.4.2. Sáu tổ hợp truyền thống

Bảng 2.12. Kết quả test của sáu mô hình cơ sở

Mã	Biểu diễn	Thuật toán	Accuracy	Precision	Recall	F1 spam	AP
B1	BoW	NB	0,9141	0,7441	0,4953	0,5947	0,7320
B2	BoW	SVM	0,9241	0,7110	0,6803	0,6953	0,7492
B3	TF-IDF	NB	0,9257	0,8402	0,5142	0,6380	0,7988
B4	TF-IDF	SVM	0,9419	0,8818	0,6278	**0,7334**	**0,8117**
B5	FastText	NB	0,8493	0,4374	0,6425	0,5204	0,5360
B6	FastText	SVM	0,9253	0,8259	0,5237	0,6409	0,7179

B4 có F1 và AP test cao nhất trong **sáu mô hình cơ sở**, nhưng Recall 0,6278 nghĩa là còn bỏ sót 354 trong 951 spam. B2 đạt Recall 0,6803 cao hơn B4 nhưng Precision thấp hơn. B5 có Accuracy 0,8493, thấp cả so với tỷ lệ ham của test; ví dụ này cho thấy embedding dày không mặc nhiên tốt hơn đặc trưng thưa trên bộ dữ liệu hiện tại. Ma trận của sáu mô hình giúp xem số lỗi thay vì chỉ nhìn F1.



Hình 2.5. Ma trận nhầm lẫn của sáu mô hình cơ sở

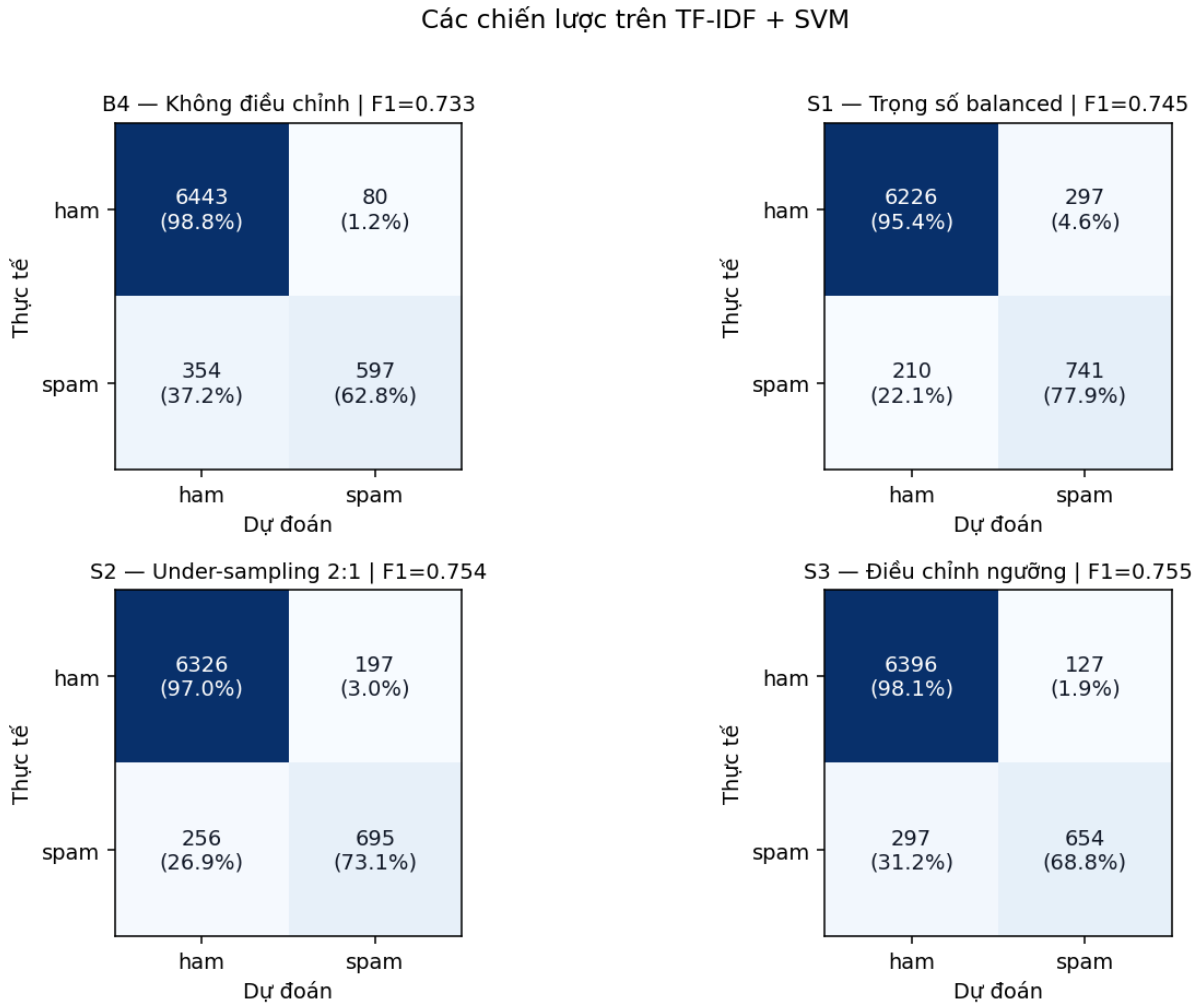
2.4.3. Tác động của ba chiến lược SVM

B4 được dùng làm đối chứng ở ngưỡng 0. S1, S2, S3 thay đổi riêng trọng số, số mẫu train hoặc ngưỡng. Bảng dưới đây là **test**, khác bảng CV ở mục 2.2.5. Chỉ số test của B4 chỉ xuất hiện như đối chứng, không phải một ứng viên thứ mười độc lập.

Bảng 2.13. Kết quả test của B4 và ba chiến lược SVM

Mã	Chiến lược	Accuracy	Precision	Recall	F1 spam	AP	FP	FN
B4	Không điều chỉnh	0,9419	**0,8818**	0,6278	0,7334	0,8117	**80**	354
S1	Balanced	0,9322	0,7139	**0,7792**	0,7451	**0,8127**	297	**210**
S2	Giảm mẫu 2:1	0,9394	0,7791	0,7308	0,7542	0,7826	197	256
S3	Chỉnh ngưỡng	**0,9433**	0,8374	0,6877	**0,7552**	0,8117	127	297

S1 tìm được nhiều spam nhất (Recall 0,7792) nhưng gắn sai 297 ham; B4 gắn sai ít ham nhất nhưng bỏ sót nhiều spam. S3 tăng Recall so với B4 từ 0,6278 lên 0,6877 và giảm FN từ 354 xuống 297, đổi lại FP tăng từ 80 lên 127. S3 đứng đầu F1 trong bốn cấu hình khảo sát, nhưng chỉ cao hơn S2 khoảng 0,0010 điểm; khoảng bootstrap của hai F1 chồng lấn. Không nên tuyên bố S3 thắng S2 một cách chắc chắn về thống kê hoặc trên mọi tập dữ liệu. S3 giữ dữ liệu train nguyên bản và có luồng suy luận đơn giản nên được chọn làm đại diện truyền thống theo tiêu chí CV F1 đã chốt.



Hình 2.6. Ma trận nhầm lẫn B4 và ba chiến lược TF-IDF + SVM

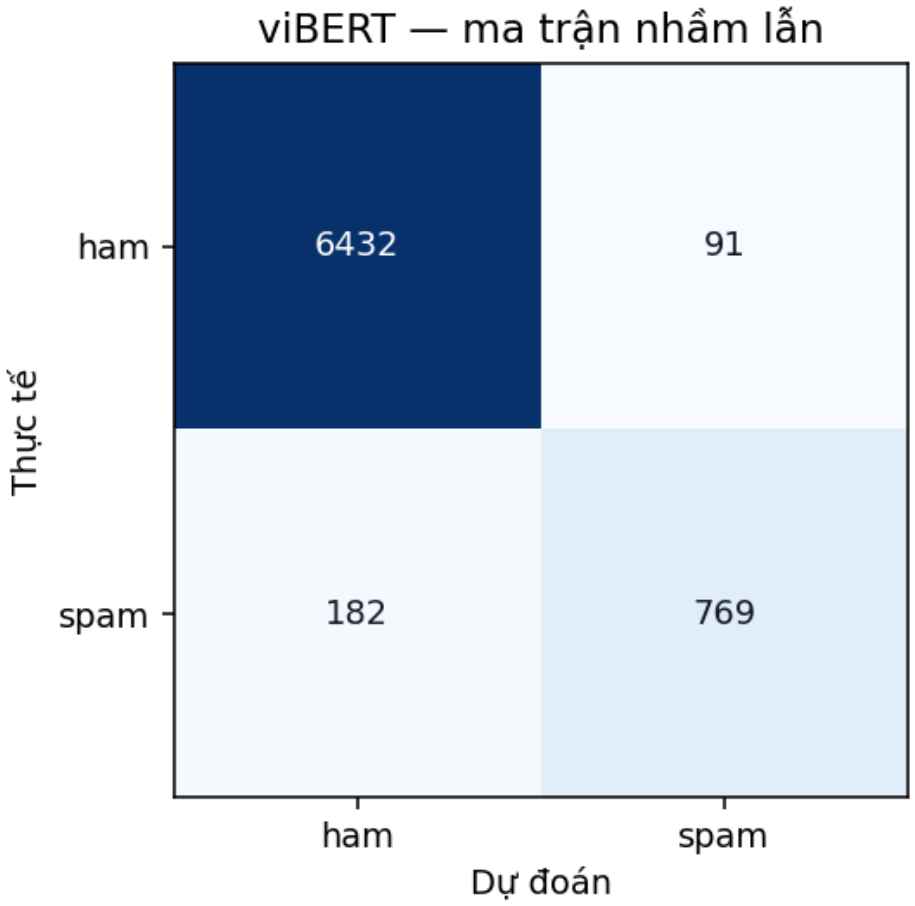
AP của B4 và S3 cùng 0,8117 vì cả hai dùng cùng một mô hình và cùng score `decision_function`; đổi ngưỡng chỉ đổi nhãn nhị phân ở một điểm vận hành, không đổi thứ tự score để tính AP. Hình PR của các chiến lược trong artifacts cho phép quan sát điều này, nhưng bảng trên đã thể hiện rõ đánh đổi chính.

2.4.4. Kết quả viBERT và đối chiếu S3

Bảng 2.14. Đối chiếu S3 và M1 trên cùng tập test

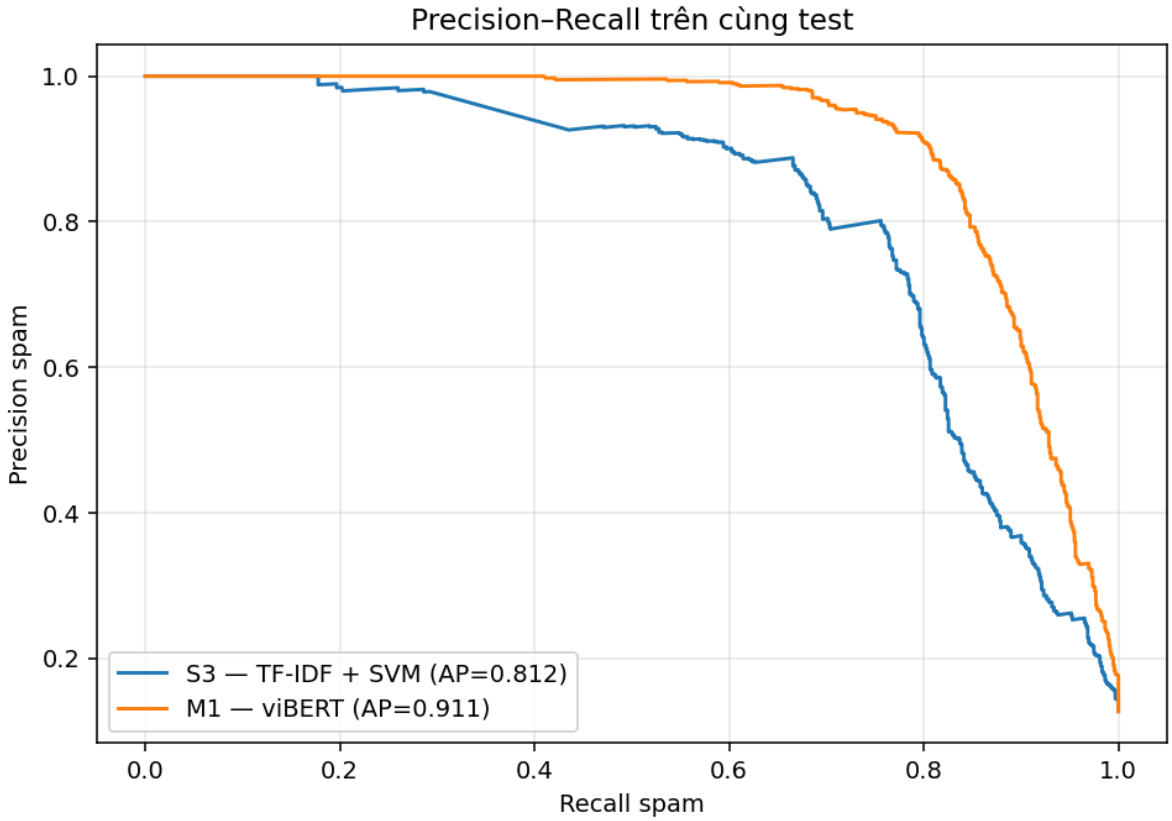
Mã	Mô hình và cấu hình	Accuracy	Precision	Recall	F1 spam	AP	FP	FN
S3	TF-IDF + SVM, ngưỡng OOF	0,9433	0,8374	0,6877	0,7552	0,8117	127	297
M1	viBERT, loss thường, 3 epoch	**0,9635**	**0,8942**	**0,8086**	**0,8493**	**0,9108**	**91**	**182**

Trên cùng 951 spam test, M1 nhận đúng 769, S3 nhận đúng 654: **M1 tìm thêm 115 spam** theo số TP. Trên cùng 6.523 ham test, M1 có 91 FP, ít hơn S3 36. F1 spam của M1 cao hơn S3 khoảng **0,0941** trên phép chia này. Đây là hiệu quả quan sát được của hai cấu hình đã lưu; chưa có thí nghiệm nhiều seed hoặc phép kiểm định ghép cặp để phát biểu về ưu thế tổng quát ngoài tập test. Ma trận của M1 là `[[6432,91],[182,769]]` theo thứ tự ham, spam.



Hình 2.7. Ma trận nhầm lẫn của viBERT trên tập test chung

Đường PR ở Hình 2.8 cho thấy M1 xếp hạng spam tốt hơn trên phần lớn miền Recall của tập test này, phù hợp AP 0,9108 so với 0,8117. Đường cong phản ánh xếp hạng theo score/xác suất; F1 trong bảng được tính tại ngưỡng đã chốt cho từng mô hình.



Hình 2.8. Đường Precision–Recall của S3 và M1 trên cùng 7.474 mẫu test

2.4.5. Phân tích lỗi và độ bất định

Đọc dự đoán theo ID cho thấy trường hợp `cmt_171` có nội dung "Zalo b", nhãn dữ liệu là spam nhưng cả S3 và M1 dự đoán ham. Một từ khóa liên hệ đơn lẻ chưa nói rõ người viết đang quảng cáo hay đang trả lời bạn học; đây là ví dụ cần ngữ cảnh/kiểm nhãn. Với `cmt_1920` ("e ib page page có ngân hàng môn này nhé"), nhãn dữ liệu ham, S3 dự đoán ham nhưng M1 dự đoán spam với xác suất khoảng 0,9011. Nội dung nhắc "page" có thể làm mô hình nghiêng về spam, song không đủ để kết luận nguyên nhân chỉ từ một mẫu. Các câu chỉ gồm dấu chấm cũng xuất hiện trong dữ liệu với nhãn spam và có thể nhận score rất cao ở M1; đây là tín hiệu cần rà nhãn trước khi triển khai tự động.

Bảng 2.15. Một số trường hợp dự đoán khó theo ID

ID	Nhãn dữ liệu	S3	M1	Ghi chú đọc lỗi
cmt_171	spam	ham	ham	"Zalo b"; thiếu ngữ cảnh
cmt_1920	ham	ham	spam	"e ib page…"; M1 FP
cmt_1051	spam	ham	spam	Chỉ có "...."; nhãn cần thẩm định

Bootstrap 500 lần của S3 cho khoảng F1 95% theo phân vị **[0,7337; 0,7776]**; của S2 là **[0,7325; 0,7758]**. Hai khoảng này mô tả dao động khi lấy mẫu lại từng dòng test, không phải bằng chứng kiểm định chênh lệch S3–S2. Project chưa tính khoảng bootstrap tương ứng cho M1, vì vậy không gán khoảng hoặc giá trị p cho so sánh S3–M1. Việc mẫu gần trùng/chung ngữ cảnh làm các dòng không hoàn toàn độc lập cũng là giới hạn của bootstrap theo dòng.

2.5. Lưu mô hình và ứng dụng bình luận mới

2.5.1. Sản phẩm lưu và cách nạp lại

Mô hình truyền thống được lưu ở `artifacts/spamham/spam_ham_ml_only.joblib`; bundle chứa pipeline TF-IDF + LinearSVC đã fit, cấu hình xử lý văn bản và ngưỡng −0,1776. Hàm `pipeline/inference/predict.py` nạp bundle, tái áp dụng tiền xử lý rồi lấy score để ra nhãn. viBERT được lưu dưới `artifacts/modern/vibert/model/` với `model.safetensors`, config, tokenizer và `inference_config.json`; `pipeline/modern/predict.py:VibertPredictor` nạp tất cả từ thư mục local, chuẩn hóa text, token hóa, chạy encoder/head rồi lấy xác suất spam. Hai mô hình đã được nạp lại và đối chiếu dự đoán với tệp kết quả lưu; quá trình nạp không huấn luyện lại.

2.5.2. Luồng suy luận

Với S3, bình luận mới đi qua `preprocess` giống train, TF-IDF vectorizer đã học tạo véc-tơ, LinearSVC trả score và quyết định spam nếu score > −0,1776. Với M1, text gốc được chuẩn hóa nhẹ, tokenizer cased tạo ID/mask, encoder/head trả logits, softmax cho xác suất spam và ngưỡng 0,5 quyết định nhãn. Không được dùng số xác suất M1 để đọc như score S3, vì hai thang đo khác nhau. Mã dưới đây rút gọn lời gọi suy luận có sẵn, không hiển thị thông tin phiên Facebook hay khóa API.

```
from pipeline.modern.predict import VibertPredictor

model = VibertPredictor("artifacts/modern/vibert/model")
result = model.predict(["Cho mình hỏi lịch thi với ạ"])
# result: text, probability_spam, du_doan
```

2.5.3. Minh họa dự đoán mới

Bảng 2.16. Minh họa suy luận viBERT trên câu mới

Văn bản đưa vào M1	Xác suất spam	Nhãn dự đoán
Cho mình hỏi lịch thi với ạ	0,0006	ham
Liên hệ 0901234567 hoặc https://example.com	0,9994	spam
.	0,9997	spam

Hai câu đầu thể hiện luồng suy luận thường gặp trong demo. Câu chỉ có "." bị M1 dự đoán spam gần như tuyệt đối, cho thấy xác suất cao không đồng nghĩa mô hình hiểu đúng ý định nếu văn bản không đủ thông tin. Bảng này là demo trên câu mới, **không thay** bảng test 7.474 bình luận. Sản phẩm hiện tại là hàm Python và notebook trình diễn; báo cáo không gọi đó là ứng dụng web hay API đã triển khai.

2.6. Thảo luận và nhận xét thực nghiệm

2.6.1. Những kết quả đạt được

Trên tập test cố định, TF-IDF + SVM là tổ hợp cơ sở tốt nhất theo F1 spam trong sáu tổ hợp. Điều chỉnh ngưỡng bằng OOF nâng Recall/F1 của B4 mà không phải thay số mẫu train; phương án có trọng số và giảm mẫu cho những điểm vận hành khác, hữu ích khi chi phí bỏ sót spam cao. viBERT sau fine-tuning đạt F1 spam 0,8493 và tìm thêm 115 spam so với S3, đồng thời giảm FP trên cùng test. Điều này đáp ứng phần mô hình hiện đại của đề, nhưng lợi ích đo được chỉ thuộc tập và giao thức hiện tại.

2.6.2. Đánh đổi và chi phí

B4 có ít FP nhất trong các chiến lược SVM nhưng bỏ sót nhiều spam. S1 giảm FN mạnh nhất nhưng tăng FP; lựa chọn thực tế phụ thuộc mức chấp nhận cảnh báo nhầm. S3 có F1 cao nhất trong nhóm chiến lược và không phải rút bớt ham để fit cuối. viBERT cho kết quả tốt hơn ở các chỉ số test chính nhưng cần GPU cho quá trình fine-tuning và có checkpoint lớn hơn bundle S3. Project đã kiểm tra nạp và suy luận CPU trên một phần mẫu; chưa đo cùng một máy, cùng batch, cùng điều kiện để kết luận định lượng về độ trễ, VRAM hoặc chi phí vận hành.

2.6.3. Giới hạn của nghiên cứu

Phạm vi nhãn có thể chứa trường hợp mơ hồ, đặc biệt câu tên riêng/dấu câu hoặc phản hồi chỉ một từ. Nhiều bình luận phụ thuộc bài viết gốc hoặc tương tác trước đó, trong khi mô hình chỉ nhận text của một comment/reply. viBERT cắt câu trên 128 token và có thể tạo `[UNK]` đối với ký tự ngoài từ vựng. Chia theo khóa văn bản không kiểm tra khả năng chuyển sang nhóm Facebook khác hoặc giai đoạn thời gian khác. Việc chọn S3 bằng CV có bước chỉnh ngưỡng trên OOF, không phải nested CV; viBERT chỉ khảo sát một split/seed và hai loss. Xác suất viBERT chưa được kiểm định/calibration; confidence gán nhãn AI cũng chưa hiệu chỉnh. Những giới hạn này ngăn việc diễn giải chỉ số test như cam kết hiệu quả khi triển khai rộng.

KẾT LUẬN

Nhóm đã xây dựng chuỗi xử lý bình luận PTIT từ dữ liệu thu thập và gán nhãn đến thực nghiệm phân loại Spam/Ham. Ba biểu diễn BoW, TF-IDF, FastText được kết hợp với Naive Bayes và SVM để tạo sáu đối chứng; TF-IDF + SVM được mở rộng bằng trọng số lớp, giảm mẫu và chỉnh ngưỡng. Nhóm cũng fine-tune viBERT từ checkpoint tiếng Việt, lưu lại trọng số và tokenizer, rồi so sánh trên đúng tập test dùng cho mô hình truyền thống.

Theo lựa chọn trên train/CV, S3 là đại diện truyền thống: F1 spam test 0,7552, Precision 0,8374, Recall 0,6877. viBERT M1 đạt F1 0,8493, Precision 0,8942, Recall 0,8086 trên cùng 7.474 mẫu. Kết quả cho thấy viBERT phù hợp với bài toán trong giao thức này; đồng thời mô hình vẫn nhầm ở những câu thiếu ngữ cảnh và dữ liệu nhãn cần tiếp tục rà soát. File mô hình đã lưu và hàm suy luận cho phép thử bình luận mới mà không cần chạy lại huấn luyện.

Hướng phát triển là rà nhãn khó với người thẩm định, thu thập thêm bình luận ở thời điểm/nhóm khác, đưa ngữ cảnh bài viết vào mô hình khi có quyền sử dụng phù hợp, đánh giá nhiều seed và test theo thời gian/nhóm. Nếu cần sử dụng thực tế, nhóm sẽ nghiên cứu hiệu chỉnh xác suất, ngưỡng theo chi phí FP/FN và xây dựng giao diện/API sau khi đánh giá thêm. Các hướng này là công việc tương lai, không phải chức năng đã có trong project.

TÀI LIỆU THAM KHẢO

[1] C. D. Manning, P. Raghavan, H. Schütze (2008), *Introduction to Information Retrieval*, Cambridge University Press. https://www-nlp.stanford.edu/IR-book/ (truy cập 06/11/2026).

[2] Argilla, *Annotate datasets*, tài liệu chính thức. https://docs.argilla.io/latest/how_to_guides/annotate/ (truy cập 06/11/2026).

[3] Unicode Consortium (2026), *Unicode Standard Annex #15: Unicode Normalization Forms*. https://www.unicode.org/reports/tr15/ (truy cập 06/11/2026).

[4] Underthesea, *word_tokenize*, tài liệu chính thức. https://undertheseanlp.github.io/underthesea/docs/next/api/word-tokenize (truy cập 06/11/2026).

[5] scikit-learn (1.6.1), *Feature extraction*, phần text feature extraction. https://scikit-learn.org/1.6/modules/feature_extraction.html (truy cập 06/11/2026).

[6] P. Bojanowski, E. Grave, A. Joulin, T. Mikolov (2017), "Enriching Word Vectors with Subword Information", *Transactions of the ACL*, 5, tr. 135–146. https://aclanthology.org/Q17-1010/; DOI: 10.1162/tacl_a_00051.

[7] scikit-learn (1.6.1), *Naive Bayes*, tài liệu chính thức. https://scikit-learn.org/1.6/modules/naive_bayes.html (truy cập 06/11/2026).

[8] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, C.-J. Lin (2008), "LIBLINEAR: A Library for Large Linear Classification", *Journal of Machine Learning Research*, 9, tr. 1871–1874. https://jmlr.org/papers/v9/fan08a.html.

[9] scikit-learn (1.6.1), *LinearSVC*, API chính thức. https://scikit-learn.org/1.6/modules/generated/sklearn.svm.LinearSVC.html (truy cập 06/11/2026).

[10] A. Vaswani và cộng sự (2017), "Attention Is All You Need", *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/1706.03762.

[11] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova (2019), "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", *NAACL-HLT*, tr. 4171–4186. https://aclanthology.org/N19-1423/; DOI: 10.18653/v1/N19-1423.

[12] T. V. Bui, T. O. Tran, P. Le-Hong (2020), "Improving Sequence Tagging for Vietnamese Text using Transformer-based Neural Models", *PACLIC 34*, tr. 13–20. https://aclanthology.org/2020.paclic-1.2/.

[13] FPT.AI, *viBERT* và checkpoint *FPTAI/vibert-base-cased*. https://github.com/fpt-corp/viBERT; https://huggingface.co/FPTAI/vibert-base-cased (truy cập 06/11/2026).

[14] Hugging Face, *BERT / BertTokenizer*, tài liệu Transformers. https://huggingface.co/docs/transformers/model_doc/bert#berttokenizer (truy cập 06/11/2026).

[15] PyTorch, *CrossEntropyLoss*, tài liệu chính thức. https://docs.pytorch.org/docs/2.15/generated/torch.nn.CrossEntropyLoss.html (truy cập 06/11/2026).

[16] I. Loshchilov, F. Hutter (2019), "Decoupled Weight Decay Regularization", *ICLR*. https://arxiv.org/abs/1711.05101.

[17] Hugging Face, *Trainer / TrainingArguments*, tài liệu Transformers 4.57.1. https://huggingface.co/docs/transformers/v4.57.1/en/main_classes/trainer (truy cập 06/11/2026).

[18] scikit-learn (1.6.1), *Cross-validation: evaluating estimator performance*. https://scikit-learn.org/1.6/modules/cross_validation.html (truy cập 06/11/2026).

[19] scikit-learn (1.6.1), *compute_class_weight*, API chính thức. https://scikit-learn.org/1.6/modules/generated/sklearn.utils.class_weight.compute_class_weight.html (truy cập 06/11/2026).

[20] imbalanced-learn, *Under-sampling*, tài liệu chính thức. https://imbalanced-learn.org/stable/under_sampling.html (truy cập 06/11/2026). Project sử dụng hàm NumPy riêng, không gọi RandomUnderSampler.

[21] scikit-learn (1.5.2), *Tuning the decision threshold for class prediction*. https://scikit-learn.org/1.5/modules/classification_threshold.html (truy cập 06/11/2026). Project tự tìm ngưỡng OOF, không gọi TunedThresholdClassifierCV.

[22] scikit-learn (1.6.1), *Metrics and scoring: quantifying the quality of predictions*. https://scikit-learn.org/1.6/modules/model_evaluation.html (truy cập 06/11/2026).

[23] B. Efron (1979), "Bootstrap Methods: Another Look at the Jackknife", *The Annals of Statistics*, 7(1), tr. 1–26. https://doi.org/10.1214/aos/1176344552.

[24] mohdtalal3, *facebook_post_comment_scraper*, GitHub. https://github.com/mohdtalal3/facebook_post_comment_scraper (truy cập 06/11/2026).

[25] p-storm, *ptit-facebook-comments-spamham*, địa chỉ dữ liệu. https://huggingface.co/datasets/p-storm/ptit-facebook-comments-spamham (truy cập 06/11/2026). Số liệu báo cáo được kiểm từ CSV v7 local.